

SION STIKOM Bali — Security Review

Tanggal: 18 Juni 2026

Reviewer: Afiq Andico, mahasiswa Program Studi Sistem Informasi

Metode: Static analysis saja (HTTP headers + HTML inspection via curl). **TIDAK ada eksploitasi, login attempt, atau active testing.**

Target: `https://sion.stikom-bali.ac.id/`

Cakupan: Halaman publik (login, forgot_password, static assets, robots.txt). Area authenticated **tidak termasuk** scope.

Ringkasan Eksekutif

Saya melakukan tinjauan keamanan ringan terhadap SION dari permukaan publik. **Tujuannya: memberi umpan balik konstruktif, bukan menuduh atau mencari kelemahan untuk dieksploitasi.**

Kesimpulan singkat: SION punya fondasi keamanan yang **cukup baik** (Cloudflare WAF, HSTS, CSRF, POST login, no reflected input di login). Yang perlu ditambah adalah **konsistensi** (header security) dan **hardening preventif** (code smell di fungsi toast, defense in depth via CSP).

Severity	Jumlah Temuan	Ringkasan
Critical	0	Tidak ada kerentanan kritis yang bisa dieksploitasi langsung dari public surface
Medium	2	Security header hilang di halaman dinamis; latent XSS code smell
Low	3	Framework CI 3 (tua); <code>autocomplete="off"</code> UX issue; default CSRF token name
Informational	1	<code>robots.txt</code> Content-Signal configuration

Tidak ada exploit path yang verified dari public surface. Semua rekomendasi adalah **hardening dan defense in depth.**

1. Yang Sudah Bagus (Positif)

Kontrol	Bukti	Catatan
Cloudflare WAF aktif	Probing ke <code>/.env</code> , <code>/.git/HEAD</code> , <code>/phpinfo.php</code> semua di-block dengan halaman "Attention Required! Cloudflare" (Ray ID dilampirkan di response)	WAF rules untuk sensitive paths berfungsi, termasuk protection terhadap common exploit scanning
HSTS aktif	<code>strict-transport-security: max-age=31536000</code>	1 tahun, kuat
Login method POST (bukan GET)	<code><form method="post" action="/validation"></code>	Kredensial tidak masuk ke URL/log. Mencegah reflected XSS via URL parameter
CSRF token pada form login	Hidden field <code>csrf_test_name</code> (32-char)	Token ada per-request, harus divalidasi server-side
Cache-Control: no-store	Header <code>Cache-Control: no-store, max-age=0, no-cache</code> di halaman login	Mencegah caching form sensitif di browser/proxy
Tidak ada GET forms	grep terhadap HTML: zero GET methods	Tidak ada attack surface untuk reflected XSS via URL
Tidak ada XSS critical patterns	Tidak ada <code>eval()</code> , <code>document.write()</code> , <code>dangerouslySetInnerHTML</code> di user-facing script	Latent risk via <code>showToast</code> dibahas di Temuan #2
x-frame-options di static assets	<code>SAMEORIGIN</code> di PDF tutorial	Clickjacking protection untuk static
x-content-type-options di static assets	<code>nosniff</code> di PDF tutorial	MIME sniffing protection untuk static
x-xss-protection di static assets	<code>1; mode=block</code> di PDF tutorial (legacy, deprecated tapi harmless)	Browser modern mengabaikannya, tetap defense in depth
Server header disamarkan	<code>Server: cloudflare</code> (bukan Apache/Nginx version)	Mengurangi info disclosure backend
External fonts lewat HTTPS	<code>fonts.googleapis.com</code> , <code>fonts.gstatic.com</code>	Privacy-friendly
Forgot password pakai 3-faktor	NIM + Tempat Lahir + Tanggal Lahir	Lebih aman dari single-factor reset

Kesimpulan: fondasi keamanan Anda sudah ada. Yang dibutuhkan adalah **konsistensi** (header juga di halaman dinamis) dan **hardening preventif** (latent XSS code smell).

2. Temuan & Rekomendasi

● TEMUAN 1: Security Headers Tidak Lengkap di Halaman Dinamis

Severity: Medium

Lokasi: Semua halaman yang di-render CodeIgniter (login, forgot_password, dashboard setelah login, dll)

Tipe: Defense in depth gap

Bukti reproduksi:

```
curl -sI https://sion.stikom-bali.ac.id/ | grep -iE "x-content-security|referrer|permissions-policy"
# (no output = no headers present)
```

Header yang HILANG di halaman dinamis (tapi ADA di static asset PDF):

Header	Nilai yang disarankan	Alasan
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src https://fonts.gstatic.com; img-src 'self' data:; connect-src 'self'; frame-ancestors 'self';	Mencegah XSS (blok inline script), data injection, clickjacking
Referrer-Policy	strict-origin-when-cross-origin	Cegah bocor URL NIM/nama ke external site
X-Content-Type-Options	nosniff	Mencegah browser tebak content type
X-Frame-Options	SAMEORIGIN (atau DENY)	Cegah clickjacking
Permissions-Policy	geolocation=(), camera=(), microphone=()	Disable fitur browser yang tidak perlu

Inkonsistensi: PDF tutorial punya header `X-Frame-Options: SAMEORIGIN` dan `X-Content-Type-Options: nosniff`, tapi halaman login (HTML) **tidak punya** header yang sama. Header ditambahin di level web server untuk folder `/assets/`, tapi **tidak untuk folder CodeIgniter** (`/`).

Fix untuk nginx:

```
add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-Frame-Options "SAMEORIGIN" always;
add_header Permissions-Policy "geolocation=(), camera=(), microphone=()" always;
```

Fix untuk Apache .htaccess :

```
<IfModule mod_headers.c>
  Header always set Content-Security-Policy "default-src 'self'; script-src 'self'; s
  Header always set Referrer-Policy "strict-origin-when-cross-origin"
  Header always set X-Content-Type-Options "nosniff"
  Header always set X-Frame-Options "SAMEORIGIN"
  Header always set Permissions-Policy "geolocation=(), camera=(), microphone=()"
</IfModule>
```

● TEMUAN 2: Latent XSS Code Smell — `showToast()` Menggunakan `innerHTML`**Severity:** Low → Medium**Lokasi:** Script block ke-5 di HTML source login page**Tipe:** Defense in depth (preventif)**Source code:**

```
function showToast(type, message) {
  const toast = document.createElement('div');
  toast.className = 'toast ' + type;
  toast.innerHTML = message; // ← XSS risk jika dipanggil dengan input tidak aman
}
```

Penemuan:

1. Fungsi `showToast()` didefinisikan di dalam `DOMContentLoaded` handler
2. **TIDAK ADA call site yang visible** — dead code saat ini
3. Container `<div id="toast-container">` ada tapi kosong
4. Saat ini: **Active risk RENDAH** (fungsi tidak dipanggil)
5. Latent: **SEDANG** — jika developer panggil `showToast('error', '<user input>')` di fitur masa depan

Fix Opsi A — Plain text (paling aman):

```
toast.textContent = message; // ganti innerHTML jadi textContent
```

Fix Opsi B — Tetap render HTML, dengan sanitasi:

```
<script src="https://cdn.jsdelivr.net/npm/dompurify@3.0.6/dist/purify.min.js"></script>
```

```
toast.innerHTML = DOMPurify.sanitize(message);
```

Rekomendasi: Opsi A (plain text). Untuk toast notification, plain text sudah cukup.

● TEMUAN 3: Kemungkinan CodeIgniter 3 Versi Lama

Severity: Low–Medium

Indikator:

1. Meta description: `<meta name="description" content="The small framework with powerful features">` — tagline default CodeIgniter
2. CSRF token name: `csrf_test_name` — default CodeIgniter 3
3. Struktur form: `action="/validation"`, `id="loginForm"` — pattern CI 3

Implikasi: CodeIgniter 3 rilis terakhir 3.1.13 (sekitar 2022). Status: maintenance-only, banyak dianggap **end-of-life**.

Rekomendasi:

1. Patch CI 3 ke versi terakhir dulu
2. Rencana migrasi ke CI 4 untuk jangka menengah
3. Atau pertimbangkan migrasi ke **Laravel** (10/11) — security defaults lebih kuat

● TEMUAN 4: `autocomplete="off"` pada Form Login

Severity: Low

Lokasi: `<form id="loginForm" ... autocomplete="off">`

`autocomplete="off"` di level form **tidak reliable** di browser modern — banyak browser (Chrome, Firefox) mengabaikannya untuk field username/password demi UX. Proteksi yang sebenarnya tidak efektif, malah rugikan UX.

Rekomendasi:

```
<form id="loginForm" method="post" action="/validation">
  <input name="username" type="text" autocomplete="username" ... >
  <input name="password" type="password" autocomplete="current-password" ... >
</form>
```

● TEMUAN 5: `csrf_test_name` — Default Token Name

Severity: Informational

CodeIgniter 3 default CSRF token name adalah `csrf_test_name`. Tidak ada custom branding / hardening.

Rekomendasi (`application/config/config.php`):

```
$config['csrf_token_name'] = 'sion_csrf_token';  
$config['csrf_cookie_name'] = 'sion_csrf_cookie';  
$config['csrf_regenerate'] = true;
```

● TEMUAN 6: `robots.txt` Content-Signal Configuration

Severity: Informational

Content-Signal: `ai-train=no` adalah standar **EU Copyright Directive 2019/790**. Legal dan tidak漏洞. Pertimbangkan tambah `ai-input=no` juga.

3. Yang TIDAK Bisa Saya Verifikasi (dan Perlu Dicek Internal)

Area	Yang perlu dicek	Cara cek
Rate limiting login	Limit attempt per NIM per waktu?	Code review di LoginController
Session cookie flags	HttpOnly, Secure, SameSite?	Browser DevTools
Password hashing	<code>password_hash()</code> modern atau MD5/SHA1?	Code review
Username enumeration	Response <code>/validation</code> beda untuk "user not found" vs "wrong password"?	Harus SAMA
Forgot password flow	Rate limit + token expiration + single-use?	Code review
Halaman authenticated XSS	Stored XSS di biodata, KRS, KHS, dll?	Perlu akses + izin

4. Cara Verifikasi Semua Temuan (oleh Tim IT)

```
# 1. Headers homepage
curl -sI https://sion.stikom-bali.ac.id/

# 2. Headers static asset
curl -sI https://sion.stikom-bali.ac.id/assets/file/PANDUAN_LOGIN_PRESMAN.pdf

# 3. Form login inspection
curl -s https://sion.stikom-bali.ac.id/ | grep -E "form|autocomplete|csrf"

# 4. showToast function
curl -s https://sion.stikom-bali.ac.id/ | grep -A 10 "function showToast"

# 5. Framework detection
curl -s https://sion.stikom-bali.ac.id/ | grep -E "csrf_test_name|small framework"

# 6. robots.txt
curl -s https://sion.stikom-bali.ac.id/robots.txt

# 7. Cloudflare WAF verification
curl -sIL https://sion.stikom-bali.ac.id/.env
```

5. Ringkasan Rekomendasi (Action Items)

Immediate (1-2 jam)

- [] Tambah 5 security header di halaman dinamis
- [] Ganti `toast.innerHTML = message` jadi `toast.textContent = message`

Short-term (1-2 minggu)

- [] Code review semua view CodeIgniter
- [] Set `$config['global_xss_filtering'] = FALSE;`
- [] Set custom CSRF token name
- [] Verifikasi session cookie flags
- [] Verifikasi password hashing

Medium-term (1-3 bulan)

- [] Patch CodeIgniter ke versi terakhir
- [] Rencana migrasi ke CI 4 atau Laravel
- [] Audit halaman authenticated untuk stored XSS
- [] Submit HSTS preload
- [] Penetration testing oleh pihak ketiga

Opsional

- [] Tambah `ai-input=no` ke Content-Signal
 - [] Ganti `autocomplete="off"` jadi eksplisit per-field
-

6. Penutup

Saya menulis laporan ini sebagai **feedback konstruktif**, bukan sebagai tuduhan atau exploit disclosure. SION secara keseluruhan sudah punya fondasi keamanan yang baik. Yang dibutuhkan adalah **konsistensi** dan **hardening preventif** — itu fix yang bisa dilakukan dalam 1-2 jam.

Jika tim IT ingin diskusi lebih lanjut atau butuh klarifikasi, saya terbuka.

— **Afiq Andico**

Mahasiswa Program Studi Sistem Informasi

STIKOM Bali

Email: afiqandico13@gmail.com

WhatsApp: 08990308936

Lampiran: Riwayat Pengujian

- **Tanggal:** 18 Juni 2026
- **Metode:** Static analysis (HTTP headers + HTML inspection via curl)
- **Eksplorasi: Tidak ada.** Tidak ada upaya login, tidak ada brute force, tidak ada probing sensitif, tidak ada XSS payload submission
- **Tools:** curl, browser DevTools
- **Coverage:** Halaman login, forgot_password, robots.txt, static assets, common admin paths
- **Out of scope:** Area authenticated, admin panel, database, server internal
- **Etika:** Mengacu pada prinsip responsible disclosure